

Rapport de Stage de Fin d'Études

Pour l'obtention du Diplôme d'Ingénieur d'État et du Master

Conception et mise en œuvre d'un pipeline automatisé de contrôle de la qualité des données : Cas des données CSP et KYC

Filière :

Réalisé par :

SAAD BENDAHOUE

MIAGE (Méthodes Informatiques
Appliquées à la Gestion des
Entreprises)
MIAGE I2A (Informatique et
Intelligence Artificielle)

Encadrant de l'École :

Mr. Marouan Skiti

Encadrants de CIH BANK :

Mr. Mohamed Agouzzal

Mr. Marouan Naoui

Année Universitaire : 2025 – 2026

Dédicaces

[Rédigez ici vos dédicaces]

Remerciements

[Rédigez ici vos remerciements]

Résumé

Dans un contexte de transformation digitale du secteur bancaire, la donnée constitue un levier stratégique essentiel pour la prise de décision, la gestion des risques et l'amélioration de la performance globale. Cependant, la qualité des données représente un enjeu majeur, car des données incomplètes, incohérentes ou erronées peuvent impacter significativement les processus métiers et les analyses.

Ce projet s'inscrit dans le cadre des initiatives de Data Governance au sein de CIH Bank. Il vise à concevoir et mettre en œuvre un pipeline automatisé de contrôle de la qualité des données, permettant d'assurer la fiabilité, la traçabilité et la cohérence des informations exploitées.

La solution proposée repose sur l'extraction et le traitement de données provenant de différentes sources, notamment le Data Lake et des fichiers structurés, en intégrant des règles de contrôle couvrant plusieurs dimensions de la qualité des données telles que la complétude, la validité, l'unicité et l'intégrité.

L'automatisation des contrôles permet d'instaurer une surveillance continue de la qualité des données, avec la production d'indicateurs (KPI) facilitant le pilotage et la prise de décision. Ce projet contribue ainsi à renforcer la gouvernance des données et à soutenir les initiatives analytiques et décisionnelles de la banque.

Abstract

In the context of digital transformation within the banking sector, data has become a key strategic asset for decision-making, risk management, and overall performance improvement. However, data quality remains a major challenge, as incomplete, inconsistent, or erroneous data can significantly impact business processes and analytical outcomes.

This project is part of the Data Governance initiatives at CIH Bank. Its objective is to design and implement an automated data quality control pipeline to ensure the reliability, traceability, and consistency of the data being used.

The proposed solution is based on processing data from multiple sources, including the Data Lake and structured files, while applying a set of rules covering key data quality dimensions such as completeness, validity, uniqueness, and integrity.

By automating these controls, the solution enables continuous monitoring of data quality and the generation of key performance indicators (KPIs), supporting efficient data governance and informed decision-making. This project therefore contributes to strengthening the organization's data management framework and enhancing its analytical capabilities.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	x
Introduction Générale	1
1 Présentation de l'organisme d'accueil	2
1.1 Présentation de CIH BANK	2
1.2 Missions de CIH BANK	2
1.3 Secteurs d'activité	3
1.4 Filiales	3
1.5 Actionnariat	3
1.6 Chiffres clés	3
1.7 Conformité	4
1.7.1 Éthique et Déontologie	4
1.7.2 Périmètre d'action du « Pôle Conformité » au niveau de CIH Bank	4
1.8 Gouvernance	5
1.9 Organigramme	6
1.9.1 Structure hiérarchique de CIH BANK	6
1.10 Data Office	8
2 Contexte, Problématique et Solution Proposée	9
2.1 Contexte général : la donnée au cœur de la banque digitale	9
2.1.1 La transformation digitale du secteur bancaire	9
2.1.2 La qualité des données : un enjeu stratégique et réglementaire	9
2.2 Le domaine Tiers : périmètre métier du projet	10
2.2.1 Définition du Référentiel Tiers	10
2.2.2 Les colonnes KYC et CSP : données critiques du Référentiel Tiers	10
2.3 Le pipeline de données existant : de NOVA à Tableau	11
2.3.1 Vue d'ensemble de l'architecture	11
2.3.2 Étape 1 — Système opérant : Oracle NOVA	12
2.3.3 Étape 2 — Ingestion RAW vers HDFS/Hive	13
2.3.4 Étape 3 — Transformation STAGING	13
2.3.5 Étape 4 — Virtualisation Dremio	13
2.3.6 Étape 5 — Vue finale CIHONE (28 colonnes)	13

2.4	Problématique	13
2.4.1	Absence de contrôle qualité automatisé sur le pipeline	13
2.4.2	Absence d'historisation et de traçabilité	14
2.4.3	Absence d'outillage pour les Data Stewards	14
2.5	Solution proposée	14
2.5.1	Vue d'ensemble de la solution	14
2.5.2	Soda Core : bibliothèque Python de Data Quality	15
2.5.3	Périmètre des contrôles : la zone RAW uniquement	15
2.5.4	Déclenchement automatique post-ingestion	15
2.5.5	Historisation et traçabilité quotidienne	15
3	Conception de la Solution	17
3.1	Architecture globale de la solution	17
3.1.1	Vue d'ensemble	17
3.1.2	Outils et technologies	18
3.2	Catalogue des règles de Data Quality	19
3.2.1	Principes de conception des règles	19
3.2.2	Règles DQ sur les colonnes KYC	20
3.2.3	Règles DQ sur les colonnes CSP	21
3.2.4	Règles DQ Pipeline (Fraîcheur)	22
3.3	Modèle de données DQ (Persistance)	22
3.3.1	Base Hive <code>dq_results</code>	22
3.3.2	Table <code>dq_tiers_scores</code> — Historique des scores	22
3.3.3	Table <code>dq_tiers_anomalies</code> — Détail des anomalies	23
3.3.4	Partitionnement et format de stockage	23
3.4	Conception du moteur DQ — Soda Core	23
3.4.1	Architecture du script <code>dq_tiers_checks.py</code>	23
3.4.2	Fichiers de configuration YAML	23
3.5	Conception de l'orchestration Airflow	24
3.5.1	Intégration dans le DAG <code>dailyow22</code>	24
3.6	Conception des vues Dremio DQ	24
3.7	Conception du dashboard Tableau	24
4	Réalisation de la Solution	26
4.1	Environnement technique de développement	26
4.1.1	Environnement Hadoop et cluster	26
4.1.2	Installation de Soda Core	27
4.2	Implémentation du moteur DQ — Soda Core	27
4.2.1	Structure des fichiers du projet DQ	27
4.2.2	Fichier de configuration YAML — Règles KYC (<code>checks_kyc.yml</code>)	27
4.2.3	Fichier de configuration YAML — Règles CSP (<code>checks_csp.yml</code>)	28
4.2.4	Script Python principal (<code>dq_tiers_checks.py</code>)	29
4.3	Création des tables Hive de persistance	31
4.4	Intégration dans le DAG Airflow <code>dailyow22</code>	32
4.4.1	Ajout de la tâche DQ	32
4.4.2	Planification et monitoring	32
4.5	Création des vues Dremio	32
4.5.1	Vue <code>v.dq_scores_history</code>	33
4.5.2	Vue <code>v.dq_kpis_summary</code>	33
4.5.3	Vue <code>v.dq_anomalies_detail</code>	33

4.6	Réalisation du dashboard Tableau	34
4.6.1	Connexion Tableau à Dremio	34
4.6.2	Onglet 1 — Vue Exécutive	34
4.6.3	Onglet 2 — Détail KYC	34
4.6.4	Onglet 3 — Détail CSP	35
4.6.5	Onglet 4 — Anomalies et Remédiation	35

Table des figures

1.1	Chiffres clés de CIH BANK (31/12/2024)	4
1.2	Gouvernance de CIH BANK	5
1.3	Organigramme de CIH BANK	6
2.1	Architecture du pipeline de données Référentiel Tiers PP Clients	12
3.1	Architecture du pipeline de Data Quality — Référentiel Tiers PP Clients .	17
3.2	Positionnement de la tâche DQ dans le DAG dailyow22	24
4.1	Dashboard Tableau — Vue Exécutive : scores DQ globaux KYC et CSP . .	34
4.2	Dashboard Tableau — Détail KYC : score par colonne et dimension DQ .	35
4.3	Dashboard Tableau — Détail CSP : taux de nullité et valeurs parasites . .	35

Liste des tableaux

1.1	Informations générales sur le groupe CIH BANK [10]	2
2.1	Colonnes KYC du Référentiel Tiers PP	11
2.2	Colonnes CSP du Référentiel Tiers PP	11
3.1	Outils et technologies de la solution DQ Tiers	18
3.2	Règles DQ — Colonnes KYC	20
3.3	Règles DQ — Colonnes CSP	21
3.4	Règles DQ — Fraîcheur et Pipeline	22
3.5	Modèle de la table <code>dq_results.dq_tiers_scores</code>	22
3.6	Modèle de la table <code>dq_results.dq_tiers_anomalies</code>	23
3.7	Vues Dremio du schéma <code>dq-tiers</code>	24
4.1	Versions des composants de l'environnement technique	26

Liste des abréviations

Abréviation	Signification
CIH	Crédit Immobilier et Hôtelier
CDO	Chief Data Officer
DWH	Data Warehouse
BI	Business Intelligence
ETL	Extract, Transform, Load
SQL	Structured Query Language
KPI	Key Performance Indicator
API	Application Programming Interface
CSV	Comma-Separated Values
ANRF	Autorité Nationale du Renseignement Financier
CNSS	Caisse Nationale de Sécurité Sociale
CSP	Catégories Socio-Professionnelles
KYC	Know Your Customer

Introduction Générale

Au cours des dernières années, le secteur bancaire a connu une transformation profonde sous l'impulsion de la digitalisation et de l'essor des technologies liées à la donnée. Dans ce contexte, les institutions financières s'appuient de plus en plus sur leurs systèmes d'information pour analyser les comportements clients, optimiser leurs processus et piloter leurs activités de manière efficace.

La donnée est ainsi devenue un actif stratégique. Toutefois, sa valeur dépend directement de sa qualité. Des données inexactes ou mal structurées peuvent engendrer des erreurs d'analyse, des décisions inadaptées et des risques importants, notamment dans des domaines sensibles tels que l'octroi de crédit ou la conformité réglementaire.

Face à ces enjeux, les organisations mettent en place des démarches de Data Governance visant à structurer, contrôler et valoriser leurs données. Parmi les axes majeurs de cette gouvernance figure la gestion de la qualité des données, qui repose sur la définition de règles de contrôle, leur automatisation et le suivi d'indicateurs de performance.

C'est dans ce cadre que s'inscrit le présent projet, réalisé au sein de CIH Bank. Il a pour objectif de concevoir et de mettre en œuvre une solution automatisée permettant de contrôler, mesurer et améliorer la qualité des données.

Le présent rapport est structuré comme suit : le premier chapitre présente l'organisme d'accueil, le deuxième expose le contexte et la problématique, le troisième décrit la conception de la solution, le quatrième détaille sa réalisation, et le dernier chapitre analyse les résultats obtenus.

Chapitre 1

Présentation de l'organisme d'accueil

Introduction

Dans ce chapitre, nous présenterons une description générale de l'organisme d'accueil, CIH BANK, d'un point de vue organisationnel. Nous allons également présenter les prestations proposées par cet organisme, ses secteurs d'activité, ainsi que quelques chiffres clés.

1.1 Présentation de CIH BANK

Le Crédit Immobilier et Hôtelier (CIH), plus communément appelé CIH BANK, est une banque marocaine dont le siège social se trouve à Casablanca. Historiquement connu pour son activité dans le secteur de l'immobilier, le CIH propose désormais des services bancaires variés à ses clients, particuliers et professionnels. [10]

CIH BANK a été fondée en 1920 sous le nom de Caisse de Prêts Immobiliers du Maroc (CPIM). En 1967, le groupe a étendu son activité au secteur hôtelier et a changé de nom pour devenir le Crédit Immobilier et Hôtelier. [10]

Création	1920
Fondateur	L'État marocain
Introduction en bourse	1967
Forme juridique	Société anonyme
Slogan	La banque de demain dès aujourd'hui
Siège social	187, Avenue Hassan II, Casablanca, Maroc
Président Directeur Général (PDG)	Lotfi SEKKAT
Société mère	Caisse de Dépôt et de Gestion
Effectif	2 181
Site web	www.cihbank.ma

TABLE 1.1 – Informations générales sur le groupe CIH BANK [10]

1.2 Missions de CIH BANK

Spécialisé dans le financement immobilier et hôtelier, le CIH est une banque à vocation particulière :

- Elle est un partenaire des pouvoirs publics en matière de financement du logement depuis 1920.
- Elle finance les promoteurs immobiliers au Maroc, par deux canaux différents :
 - En amont : Par le financement des promoteurs publics et privés (acquisition et viabilisation des terrains ainsi que la construction des programmes immobiliers).
 - En aval : Par l’octroi de crédits au logement pour les particuliers acquéreurs, sans oublier les services et produits proposés par la banque pour sa clientèle.

1.3 Secteurs d’activité

Le groupe CIH BANK et ses filiales offrent une vaste gamme de produits et de services à ses clients, particuliers et professionnels :

- Services bancaires ,
- Prêts aux particuliers (prêts immobiliers, crédits à la consommation, etc.) ,
- Financement d’entreprises (prêts d’investissement et d’exploitation, crédit-bail, etc.) ,
- Assurance ,
- Salle des marchés.

1.4 Filiales

Le groupe CIH BANK est constitué de plusieurs filiales :

- SOFAC : entreprise spécialisée dans les solutions de crédit ,
- SOFASSUR : entreprise spécialisée dans les services d’assurance ,
- Maroc Leasing : entreprise de leasing qui propose des solutions de financement pour les véhicules, le BTP, l’immobilier et la santé [6] ,
- CIH Courtage : courtier en assurance ,
- Umnia Bank : banque participative ,
- AJAR INVEST : société de gestion des Organismes de Placement Collectif Immobilier (OPCI), dont CIH BANK détient 40 % des parts [1] .

1.5 Actionnariat

L’actionnaire principal (55,66 %) du groupe CIH BANK est Massira Capital Management. De plus, 16,37 % des actions de l’entreprise sont flottantes en bourse. Le reste des parts de la banque est réparti entre Atlanta Sanad (11,61 %), Caisse de Dépôt et de Gestion - CDG (6,69 %), Régime Collectif d’Allocation de Retraite - RCAR (5,21 %), le personnel de la banque (4,36 %) et Groupe Holmarcom (0,12 %). [4]

1.6 Chiffres clés

La figure 1.1 présente les chiffres clés de CIH BANK.

Les chiffres clés de CIH BANK reflètent la situation financière et opérationnelle de l’institution. Avec un capital social de 3,149 milliards de dirhams et un total bilan de 141 milliards de dirhams, la banque dispose d’une base financière significative. L’encours des dépôts s’élève à 84,5 milliards de dirhams, tandis que l’encours global de crédit atteint 101,2 milliards de dirhams. Le produit net bancaire est de 4 739,5 millions de dirhams,

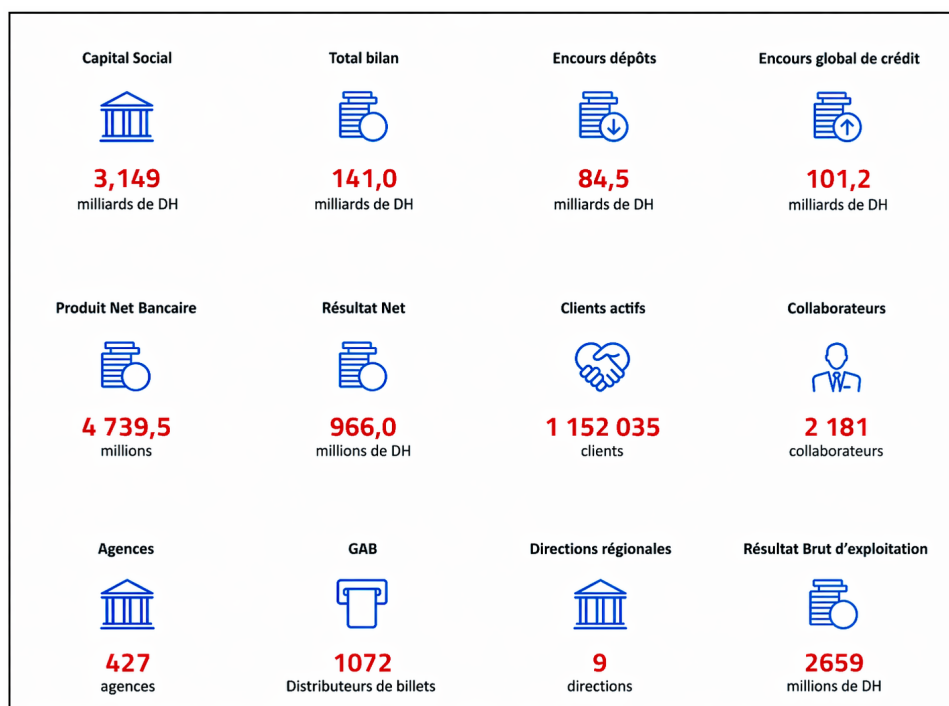


FIGURE 1.1 – Chiffres clés de CIH BANK (31/12/2024)

avec un résultat net de 966 millions de dirhams et un résultat brut d'exploitation de 2 659 millions de dirhams. La banque compte plus de 1,15 million de clients actifs, 2 181 collaborateurs, 427 agences, 1 072 guichets automatiques et 9 directions régionales. Ces indicateurs fournissent une vue d'ensemble de l'activité, de la structure et des résultats financiers de l'institution.

1.7 Conformité

La fonction Conformité est chargée du suivi du risque de non-conformité. Ledit risque représente le risque de réputation, de pertes financières et de sanctions en raison de manquement aux dispositions législatives et réglementaires, des normes et pratiques applicables à ses activités ou des principes issus du Code d'Éthique & de Déontologie. [4]

1.7.1 Éthique et Déontologie

CIH Bank dispose d'un Code d'Éthique & de Déontologie mis à jour en décembre 2021. Ce dernier regroupe un ensemble de valeurs éthiques et de règles régissant les relations de la banque avec ses collaborateurs et ses partenaires externes (Clients, fournisseurs, administrateurs, presse, etc.). [4]

Le code d'Éthique & de Déontologie reprend les principes à respecter par le personnel fondé sur les valeurs de l'Intégrité, du Professionnalisme, de la Loyauté, de la Méritocratie et de la Solidarité. [4]

1.7.2 Périmètre d'action du « Pôle Conformité » au niveau de CIH Bank

- La sécurité financière ,
- Connaissance clientèle (KYC) et devoir de vigilance ,

- Respect des sanctions internationales et Embargos ,
- Lutte contre la Fraude ,
- La protection de données à caractère personnel ,
- FATCA (Foreign Account Tax Compliance Act) ,
- Système de Management Anti-Corruption (SMAC) ,
- Le Dispositif Externe d'Alerte Éthique .

1.8 Gouvernance

La gouvernance de CIH BANK implique des membres exécutifs et des administrateurs indépendants... La figure 1.2 illustre la structure de gouvernance.

- Comité de Nomination ,
- Comité des Grands Engagements ,
- Comité d'Audit ,
- Comité des Risques .



FIGURE 1.2 – Gouvernance de CIH BANK

1.9 Organigramme

L'organigramme de CIH BANK est présenté dans la figure 1.3.

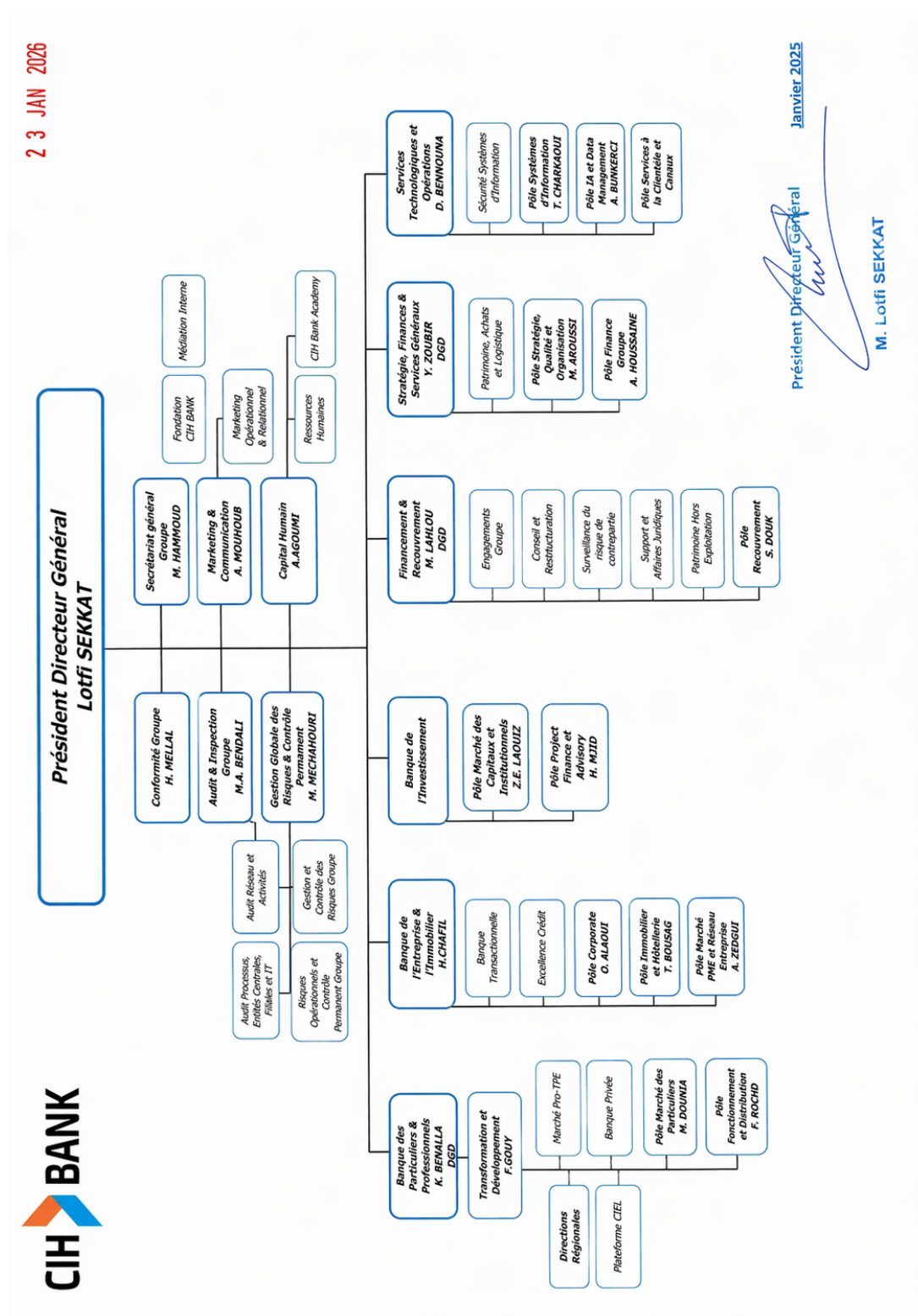


FIGURE 1.3 – Organigramme de CIH BANK

1.9.1 Structure hiérarchique de CIH BANK

La structure hiérarchique de CIH BANK est organisée comme suit :

1. Niveau stratégique

- Président Directeur Général (PDG) : Lotfi SEKKAT.
- Fonctions de support stratégique et de contrôle :
 - Secrétariat Général : M. HAMMOUD ;
 - Conformité Groupe : H. MELLAL ;
 - Marketing & Communication : A. MOUHOUB ;
 - Audit & Inspection Groupe : M. A. BENDALI ;
 - Capital Humain & Services Généraux : F. GOUY ;
 - Gestion des Risques & Contrôle Permanent Groupe : M. MECHAHOURI ;
 - Stratégie, Finance & Développement : Y. ZOUBIR.

2. Fonctions rattachées

- Secrétariat Général :
 - Fondation CIH BANK ;
 - Médiation interne.
- Capital Humain & Services Généraux :
 - Ressources humaines ;
 - CIH Bank Academy.
- Marketing & Communication :
 - Marketing digital et multicanal.

3. Pôles métiers

- Banque des Particuliers et Professionnels : K. BENALLA
 - Directions régionales ;
 - Marché Pro & TPE ;
 - Banque Privée ;
 - Pôle Fonctionnement et Distribution : F. ROCHD ;
 - Pôle Marché des Particuliers : M. DOUNIA.
- Banque de l'Entreprise et de l'Immobilier :
 - Banque Transactionnelle ;
 - Excellence Crédit ;
 - Pôle Corporate : O. ALAOUI ;
 - Pôle Immobilier & Hôtellerie : T. BOUSAG.
- Banque de l'Investissement : M. MIMOUNI
 - Marché des Capitaux ;
 - Corporate Finance ;
 - International.
- Financement & Recouvrement : M. LAHLOU
 - Engagements ;
 - Surveillance du risque de contrepartie ;
 - Support & Affaires Juridiques ;
 - Patrimoine Hors Exploitation ;
 - Pôle Recouvrement : S. DOUK.
- Services à la Clientèle & Réseaux : A. AGOUMI
 - Gestion des Comptes & Flux Domestiques ;
 - Flux internationaux ;
 - Traitement des Prêts ;
 - Factoring & Leasing.

4. Fonctions technologiques et data

- Services Technologiques et Opérations : D. BENNOUNA
- Sécurité des Systèmes d'Information ;
- Qualité ;
- Pôle Systèmes d'Information : T. CHARKAOUI ;
- Data Management & Intelligence Artificielle :
 - Data Office ;

1.10 Data Office

Dans le cadre de notre stage, nous avons intégré le Data Office de CIH BANK, une entité dédiée à la gestion, au traitement et à la valorisation des données de la banque.

Le Data Office est aujourd'hui rattaché au pôle Services Technologiques et Opérations, plus précisément au sein de la direction Data Management et Intelligence Artificielle. Cette organisation reflète l'importance stratégique accordée à la donnée dans le pilotage des activités de la banque.

Cette entité regroupe plusieurs profils complémentaires intervenant sur l'ensemble du cycle de vie de la donnée, notamment :

- Les Data Engineers, en charge de l'ingestion, du traitement et de la mise à disposition des données ,
- Les Data Analysts, responsables de l'analyse des données et de la production d'indicateurs décisionnels ,
- Les Data Scientists, spécialisés dans la modélisation avancée et l'analyse prédictive ,
- L'équipe Data Management, qui assure la gouvernance, la qualité et la structuration des données .

Les données sont collectées à partir des différents systèmes opérationnels de la banque, puis intégrées et centralisées au sein d'un Data Lake. Elles sont ensuite transformées et mises à disposition pour répondre aux besoins analytiques, réglementaires et métiers.

Dans ce cadre, le Data Office joue un rôle central dans la mise en place des pratiques de gestion de la qualité des données.

Conclusion

Pour conclure, CIH BANK est une institution bancaire marocaine proposant une gamme de services financiers, incluant le financement immobilier, les services bancaires classiques, l'assurance et l'investissement. Elle intègre la digitalisation dans son offre et applique des normes de conformité et de gestion des risques. Son activité couvre divers segments, allant des particuliers aux entreprises, avec une présence sur le marché national.

Chapitre 2

Contexte, Problématique et Solution Proposée

Introduction

Ce chapitre présente le contexte général dans lequel s’inscrit ce projet de fin d’études, les enjeux de la qualité des données dans le secteur bancaire, ainsi que la problématique spécifique rencontrée au sein de CIH Bank. Nous décrirons ensuite l’architecture du pipeline de données existant avant de présenter la solution retenue et les outils mobilisés.

Note de périmètre : les contrôles de Data Quality développés dans ce projet s’appliquent exclusivement à la **zone RAW** du Data Lake. Cette zone est la copie exacte et non transformée des données issues du système opérant Oracle NOVA. Contrôler la qualité à ce niveau revient à mesurer directement la qualité des données à la source, avant toute transformation.

2.1 Contexte général : la donnée au cœur de la banque digitale

2.1.1 La transformation digitale du secteur bancaire

Le secteur bancaire connaît depuis plusieurs années une transformation profonde, portée par la digitalisation des services, l’essor du big data et l’émergence de l’intelligence artificielle. Dans ce contexte, les institutions financières s’appuient de plus en plus sur leurs systèmes d’information pour analyser les comportements clients, optimiser leurs processus internes et piloter leurs activités de manière efficace.

La donnée est ainsi devenue un actif stratégique de premier plan. Pour CIH Bank, les données clients constituent le socle de nombreuses activités critiques : l’octroi de crédit, la segmentation commerciale, la conformité réglementaire (KYC, FATCA, LAB-FT) et le pilotage de la performance. La fiabilité de ces décisions dépend directement de la qualité des données sur lesquelles elles reposent.

2.1.2 La qualité des données : un enjeu stratégique et réglementaire

La qualité des données (*Data Quality*) est la mesure dans laquelle les données sont aptes à satisfaire les besoins pour lesquels elles sont utilisées. Elle s’évalue selon plusieurs dimensions reconnues par le standard DAMA (Data Management Association) [REF] :

- **Complétude** : les champs obligatoires sont-ils renseignés ?
- **Validité** : les valeurs respectent-elles les formats et référentiels autorisés ?
- **Cohérence** : les informations inter-colonnes sont-elles logiquement compatibles ?
- **Fraîcheur** : les données sont-elles à jour après chaque cycle de chargement ?
- **Unicité** : existe-t-il des doublons sur les identifiants ou les enregistrements ?

Dans le domaine bancaire, une mauvaise qualité des données peut avoir des conséquences graves :

- **Risques réglementaires** : des données KYC incomplètes ou incohérentes exposent la banque à des sanctions de Bank Al-Maghrib (BAM) et des organismes de conformité internationaux.
- **Risques métiers** : des données socio-professionnelles (CSP) erronées faussent les modèles de scoring crédit et la segmentation commerciale.
- **Risques opérationnels** : des anomalies non détectées se propagent dans l'ensemble du système d'information, contaminant les rapports de reporting et les tableaux de bord décisionnels.

2.2 Le domaine Tiers : périmètre métier du projet

2.2.1 Définition du Référentiel Tiers

Le **Référentiel Tiers** constitue le référentiel central de CIH Bank regroupant l'ensemble des informations relatives aux personnes physiques (PP) et morales (PM) en relation avec la banque. Ce référentiel est la source de vérité pour toutes les applications métiers : les crédits, la comptabilité, le reporting réglementaire et les outils de CRM.

Notre projet se concentre sur le périmètre des **Personnes Physiques (PP) Clients**, qui regroupe l'ensemble des clients particuliers et MRE (Marocains Résidant à l'Étranger) actifs de CIH Bank, soit plus de 1,15 million de clients. Le filtre appliqué en production est : `customer_type = 1` et `code_marche IN (1, 8)`.

2.2.2 Les colonnes KYC et CSP : données critiques du Référentiel Tiers

Les colonnes ciblées par notre projet de Data Quality sont réparties en deux grands domaines :

Données KYC (*Know Your Customer*)

Les données KYC sont les informations d'identification réglementaires obligatoires pour chaque client. Elles sont issues de la table `CCM_NPDESCRIPTION` du système Oracle NOVA et comprennent notamment :

Colonne NOVA	Colonne CIHONE	Rôle métier
NOM	nom	Nom de famille du client
PRENOM	prenom	Prénom du client
DATE_NAISSANCE	date_de_naissance	Date de naissance
CIVILITE	civilite	Civilité : M., MME, MLLE
GENRE	genre	Genre : M ou F
CODE_NATIONALITE	code_pays_de_nationalite	Code pays de nationalité
CODE_PAYS_RESIDENCE	code_pays_de_residence	Pays de résidence
IDENTIFIANT_FATCA	identifiant_fatca	Numéro FATCA (clients US)
DATE_IMPRESSION_KYC	—	Date impression fiche KYC

TABLE 2.1 – Colonnes KYC du Référentiel Tiers PP

Données CSP (*Catégorie Socio-Professionnelle*)

Les données CSP décrivent la situation professionnelle du client. Elles proviennent de la table **CCM_JOB** et comprennent :

Colonne NOVA	Colonne CIHONE	Rôle métier
CODE_SECTEUR_PRO	code_secteur_professionnel	Code secteur (ex : 46=Agri-culture)
CODE_PROFESSION	code_profession	Code profession détaillée
CODE_CSP	code_categorie_socio_professionnelle	Catégorie socio-professionnelle
SITUATION_PRO	situation_professionnelle	SAL, FON, TRI, AUT, EMP, NDE
EMPLOYEUR	employeur	Nom de l'employeur
PPE	—	Personne Politiquement Exposée
NUM_CNSS	—	Numéro de sécurité sociale

TABLE 2.2 – Colonnes CSP du Référentiel Tiers PP

2.3 Le pipeline de données existant : de NOVA à Tableau

2.3.1 Vue d'ensemble de l'architecture

Les données du Référentiel Tiers suivent un parcours en plusieurs étapes, depuis le système opérant Oracle NOVA jusqu'aux outils de visualisation, en passant par un Data Lake Hadoop. Le schéma ci-dessous décrit l'architecture globale de ce flux de bout en bout.

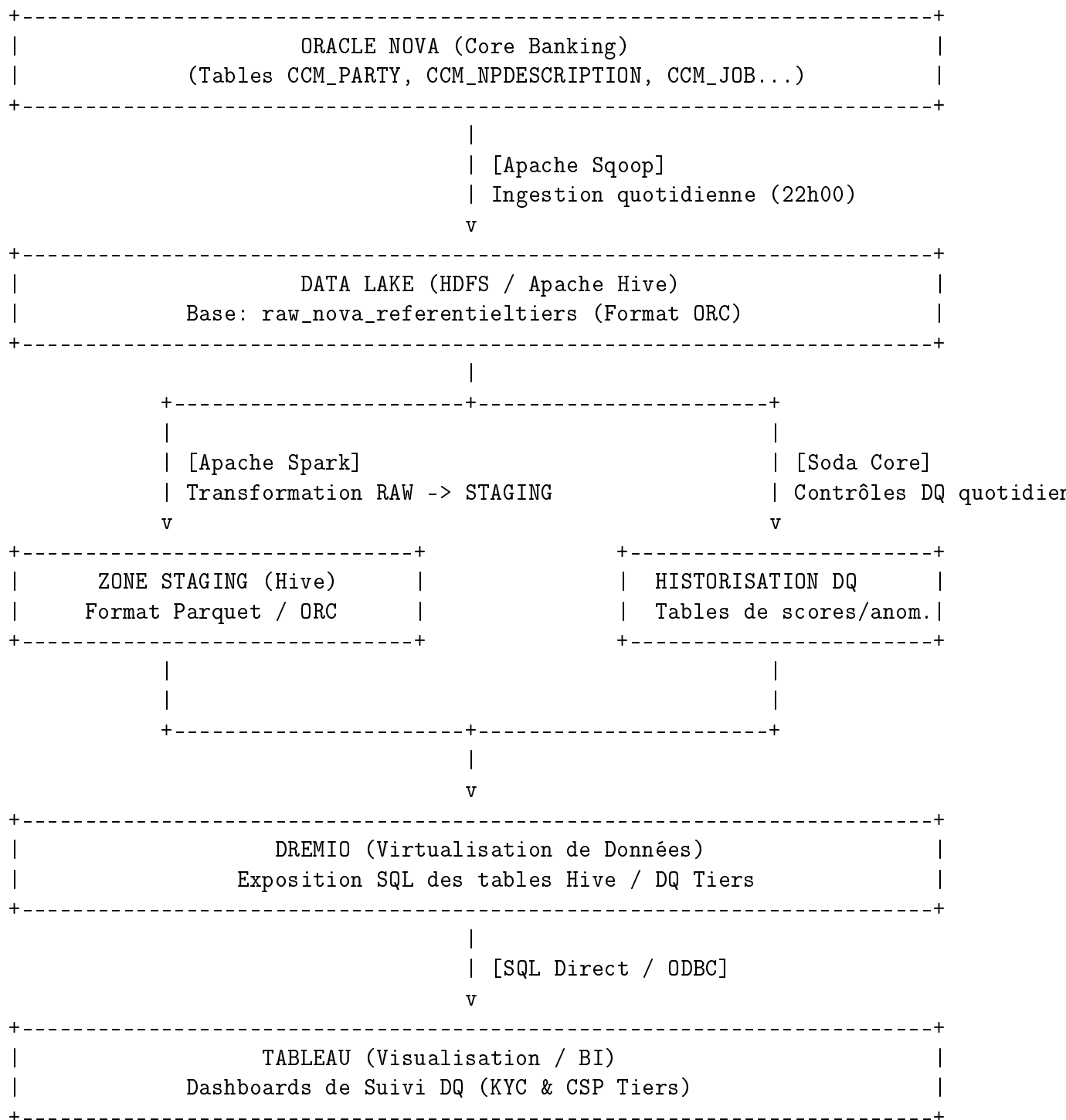


FIGURE 2.1 – Architecture du pipeline de données Référentiel Tiers PP Clients

2.3.2 Étape 1 — Système opérant : Oracle NOVA

La source de données est le système de *Core Banking Oracle NOVA*, qui héberge le schéma **REFERENTIELTIERS**. Les tables sources principales utilisées sont :

- **CCM_PARTY** : informations générales du tiers (identifiant, numéro client, marché, état)
- **CCM_NATURALPERSON** : lien entre le tiers et sa description de personne physique
- **CCM_NPDESCRIPTION** : données d'état civil et KYC (nom, prénom, date de naissance, nationalité, FATCA)
- **CCM_JOB** : données socio-professionnelles CSP (secteur, profession, CSP, employeur)

- CCM_PORTFOLIO : portefeuille commercial (gestionnaire associé)
- CCM_CONTEXT : contexte de création du dossier client (agence, utilisateur)

2.3.3 Étape 2 — Ingestion RAW vers HDFS/Hive

Le transfert quotidien des données depuis Oracle NOVA vers le Data Lake est orchestré par **Apache Airflow**. Le DAG principal du domaine Tiers est **dailyow22**, qui s'exécute chaque soir à **22h00**. Il utilise **Apache Sqoop** en mode *OverWrite* (OW) Full pour importer l'ensemble des tables CCM_* vers la base Hive **novatiers**, dans le répertoire HDFS /data/raw/novatiers/, au format ORC.

Un second DAG, **nova_referentietiers_clients**, alimente en parallèle la base Hive **raw_nova_referentietiers**, qui est la base **directement consommée par Dremio** pour la construction des vues analytiques.

2.3.4 Étape 3 — Transformation STAGING

Un job **Apache Spark** (DAG **referentietiers_ingestion**, script **staging_novareferentietiers**) transforme et charge les données depuis la zone RAW vers la zone STAGING, dans la base Hive **staging_nova_referentietiers**, au format Parquet/ORC.

2.3.5 Étape 4 — Virtualisation Dremio

Dremio expose les données de staging via **10 vues virtuelles** dans le schéma **staging-nova-refere**

- v.Customers (source : CCM_PARTY)
- v.natural_person_description (source : CCM_NPDESCRIPTION)
- v.customer_job (source : CCM_JOB)
- v.portfolio (source : CCM_PORTFOLIO)
- v.customer_creation_context (source : CCM_CONTEXT)
- v.customer_segmentation et v.customer_segmentation_members
- v.operational_posts (source : schéma staging-e_params)

2.3.6 Étape 5 — Vue finale CIHONE (28 colonnes)

La vue analytique finale **CIHONE** résulte d'une jointure entre les 10 vues Dremio, filtrée sur **customer_type=1** et **code_marche IN (1,8)**, pour produire 28 colonnes décrivant chaque client PP actif de CIH Bank. Cette vue alimente les outils de reporting (Tableau) et les analyses métiers.

2.4 Problématique

2.4.1 Absence de contrôle qualité automatisé sur le pipeline

Bien que ce pipeline de données soit robuste et opérationnel, il ne dispose d'aucun mécanisme automatisé de contrôle de la qualité des données. Les données transitent quotidiennement depuis Oracle NOVA vers Dremio **sans aucune validation intermédiaire** sur les colonnes critiques KYC et CSP.

Cette absence de contrôle génère plusieurs problèmes identifiés dans le domaine Tiers :

- **Incohérences genre / civilité** : des clients de genre masculin associés à une civilité féminine (MME, MLLE) ou inversement, signalant une saisie erronée dans NOVA.

- **Données CSP massivement nulles ou parasites** : les colonnes `professional_sector`, `occupation` et `socioprofessional_category` présentent des taux de valeurs nulles significatifs. Le champ `employer` contient de nombreuses valeurs parasites non normalisées telles que « AUTRE », « LUI MEME » ou « BIEN VU ».
- **Dates de naissance approximatives** : une proportion notable de clients enregistre une date de naissance au 1er janvier, signalant une saisie par défaut faute d'information précise.
- **Non-conformité FATCA** : des clients de nationalité américaine ne disposent d'aucun identifiant FATCA, exposant la banque à un risque réglementaire critique.
- **Risque de fraîcheur** : aucune alerte ne se déclenche si le batch `dailyow22` échoue ou se termine en retard, laissant Dremio exposer des données potentiellement obsolètes.

2.4.2 Absence d'historisation et de traçabilité

Les anomalies détectées manuellement ne sont ni tracées ni historisées. Il est donc impossible de mesurer l'évolution de la qualité des données dans le temps, d'identifier des tendances dégradantes ou de prouver une amélioration après une action corrective menée par les Data Stewards dans NOVA.

2.4.3 Absence d'outillage pour les Data Stewards

Les **Data Stewards** et **Data Owners** du domaine Tiers n'ont pas accès à un outil centralisé leur permettant de visualiser les anomalies, de les prioriser par criticité et de suivre l'avancement des corrections. La remédiation est aujourd'hui réactive et non structurée.

2.5 Solution proposée

2.5.1 Vue d'ensemble de la solution

Pour répondre à ces problématiques, nous proposons de concevoir et de mettre en œuvre un **pipeline automatisé de Data Quality (DQ)** déclenché quotidiennement après chaque exécution du DAG `dailyow22`, basé sur la bibliothèque Python **Soda Core**. La solution s'articule autour de quatre composantes majeures :

1. **Moteur de contrôle DQ (Soda Core)** : un script Python exécutant des règles de qualité paramétrables sur les tables Hive des zones RAW et STAGING du domaine Tiers.
2. **Couche de persistance (HDFS/Hive)** : des tables Hive dédiées pour historiser quotidiennement les scores DQ et le détail des anomalies détectées.
3. **Exposition analytique (Dremio)** : des vues virtuelles Dremio exposant les résultats DQ historisés pour les outils de consommation.
4. **Dashboard de suivi (Tableau)** : un tableau de bord interactif permettant aux Data Stewards et Data Owners de visualiser les KPIs de qualité, l'évolution temporelle des scores et la liste priorisée des anomalies à corriger.

2.5.2 Soda Core : bibliothèque Python de Data Quality

Soda Core est une bibliothèque Python open source dédiée au contrôle de la qualité des données. Elle permet de définir des règles de qualité (appelées *checks*) dans des fichiers de configuration YAML, de les exécuter sur différentes sources de données (Hive, Spark, PostgreSQL, Snowflake, etc.) et de collecter les résultats de manière programmatique. [REF]

Les avantages de Soda Core pour notre contexte sont les suivants :

- **Compatibilité Hive / Spark** : Soda Core dispose d'un connecteur natif compatible avec l'environnement Hadoop de CIH Bank (`soda-core-spark`).
- **Configuration déclarative** : les règles DQ sont définies en YAML, ce qui facilite leur maintenance et leur évolution sans modifier le code Python.
- **Intégration Airflow** : Soda Core s'intègre nativement dans les DAGs Airflow via un opérateur Python, permettant de le déclencher automatiquement après chaque ingestion.
- **Résultats programmatiques** : les résultats des checks (valeur calculée, seuil, statut) sont accessibles en Python, ce qui permet de les persister facilement dans des tables Hive.

2.5.3 Périmètre des contrôles : la zone RAW uniquement

Le choix de cibler la **zone RAW** du Data Lake comme unique point de contrôle est délibéré et justifié par plusieurs raisons :

- **Fidélité à la source** : la zone RAW (base Hive `raw_nova_referentietiers`, chemin HDFS `/data/raw/novatiers/`) est la copie exacte et non transformée des tables `CCM_*` issues d'Oracle NOVA. Contrôler la qualité sur RAW revient à mesurer directement la qualité du système opérant.
- **Isolation des responsabilités** : en contrôlant avant la transformation STAGING, on distingue clairement les anomalies issues du système source (responsabilité des Data Stewards Tiers) des éventuels artefacts introduits par les scripts de transformation.
- **Remédiation à la source** : les anomalies détectées sur RAW peuvent être remontées directement aux équipes saisie des agences pour correction dans NOVA, sans ambiguïté sur leur origine.

2.5.4 Déclenchement automatique post-ingestion

La solution s'insère dans le workflow Airflow existant. Une **tâche Python dédiée** (`run_dq_tiers_checks`) est ajoutée dans le DAG `dailyow22`, en dépendance de toutes les tâches d'ingestion Sqoop des tables `CCM_*`. Ainsi, les contrôles DQ se lancent **automatiquement chaque soir à 22h**, dès que les données RAW sont disponibles, sans intervention manuelle.

2.5.5 Historisation et traçabilité quotidienne

Les résultats de chaque exécution sont persistés dans deux tables Hive partitionnées par date (`run_date`) dans une base dédiée `dq_results` :

- `dq_tiers_scores` : enregistre, pour chaque combinaison table/colonne/règle, la valeur calculée, le seuil configuré, le statut (OK/KO) et le score DQ normalisé sur 100.

- `dq_tiers_anomalies` : enregistre le détail des enregistrements clients (`identifier`, `radical`) en situation d'anomalie, avec la description de la règle violée et la criticité associée.

Cette historisation permet de mesurer l'**évolution temporelle** de la qualité des données à la source et de quantifier l'impact des corrections effectuées dans NOVA par les Data Stewards.

Conclusion

Ce chapitre a présenté le contexte bancaire dans lequel s'inscrit ce projet, les enjeux critiques de la qualité des données KYC et CSP sur le Référentiel Tiers de CIH Bank, ainsi que les limites du pipeline de données actuel en matière de contrôle DQ. La solution proposée — un pipeline automatisé basé sur Soda Core, orchestré par Airflow, avec persistance dans Hive et restitution dans Tableau — répond directement à ces problématiques. Le chapitre suivant détaille la conception de cette solution.

Chapitre 3

Conception de la Solution

Introduction

Ce chapitre présente la conception détaillée du pipeline de Data Quality sur le Référentiel Tiers de CIH Bank. Les contrôles de qualité s'appliquent **exclusivement à la zone RAW** du Data Lake, qui constitue la copie exacte et fidèle des tables CCM_* issues du système opérant Oracle NOVA. Nous définirons l'architecture globale de la solution, le modèle de données pour la persistance des résultats DQ, le catalogue complet des règles de contrôle, ainsi que la conception des composants d'orchestration, d'exposition et de visualisation.

3.1 Architecture globale de la solution

3.1.1 Vue d'ensemble

La solution de Data Quality s'insère dans le pipeline existant du domaine Tiers comme une **couche transversale de contrôle** ciblant exclusivement la **zone RAW**. La logique est simple : la zone RAW est la copie conforme du système opérant Oracle NOVA. Y détecter des anomalies équivaut à mesurer la qualité réelle des données à la source, avant toute transformation. La figure 3.1 illustre l'architecture globale de bout en bout.

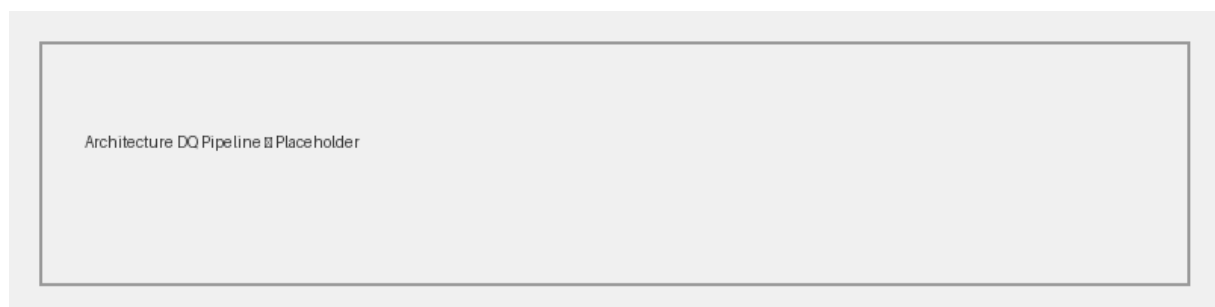


FIGURE 3.1 – Architecture du pipeline de Data Quality — Référentiel Tiers PP Clients

La solution comprend **cinq composantes principales** :

1. **Source** : Oracle NOVA (Core Banking) — inchangée
2. **Ingestion RAW** : DAG Airflow dailyow22 (22h) — inchangée
3. **Moteur DQ** : Script Python Soda Core (`dq_tiers_checks.py`) exécuté **sur la zone RAW** — **nouveau**

4. **Persistance** : Tables Hive de résultats DQ (base `dq_results`) — **nouveau**
5. **Exposition & Visualisation** : Vues Dremio `dq-tiers` + Dashboard Tableau — **nouveau**

3.1.2 Outils et technologies

Le tableau 3.1 présente les outils et technologies retenus pour la mise en œuvre de la solution.

Outil	Rôle dans la solution	Justification
Oracle NOVA	Système opérant source	Core Banking CIH Bank
Apache Sqoop	Ingestion Oracle → HDFS	Pipeline existant
Apache Airflow	Orchestration des DAGs	Pipeline existant — ajout d'une tâche DQ
Apache Hive / HDFS	Stockage RAW et STAGING	Data Lake CIH Bank
Apache Spark	Transformation STAGING	Pipeline existant
Soda Core (Python)	Moteur de contrôle DQ	Open source, compatible Spark/Hive, config YAML
Dremio	Virtualisation et exposition SQL	Data Virtualization existante
Tableau	Dashboard de visualisation DQ	Outil de reporting standard CIH Bank

TABLE 3.1 – Outils et technologies de la solution DQ Tiers



Oracle NOVA — Système opérant (Core Banking)

Système source des données du Référentiel Tiers. Héberge les tables `CCM_PARTY`, `CCM_NPDDESCRIPTION`, `CCM_JOB` contenant les informations KYC et CSP des clients PP.



Apache Sqoop — Ingestion batch Oracle vers HDFS

Outil de transfert massif de données entre bases relationnelles et Hadoop. Utilisé en mode *OverWrite* (OW) Full pour charger quotidiennement les tables `CCM_*` vers la zone RAW Hive `raw_nova_referentieltiers`.



Apache Airflow — Orchestration des pipelines de données

Plateforme d'orchestration de workflows. Le DAG `dailyow22` coordonne les tâches d'ingestion Sqoop chaque soir à 22h00. La tâche DQ Soda Core est ajoutée à ce DAG en aval de l'ingestion.



HDFS / Apache Hive — Data Lake CIH Bank

Système de stockage distribué (HDFS) et moteur de requêtage SQL (Hive). Les données RAW sont stockées dans `/data/raw/novatiers/` au format ORC. C'est sur ces tables RAW que les contrôles Soda Core sont exécutés.



Apache Spark — Moteur de transformation RAW vers STAGING

Moteur de traitement distribué en mémoire. Transforme et charge les données de la zone RAW vers la zone STAGING (`staging_nova_referentietiers`). Mobilisé également par Soda Core via le connecteur `soda-core-spark`.



Soda Core — Moteur de Data Quality (Python)

Bibliothèque Python open source de contrôle de la qualité des données. Les règles DQ sont définies en YAML (`checks_kyc.yml`, `checks_csp.yml`) et exécutées sur les tables RAW Hive. Les résultats sont persistés dans Hive pour historisation quotidienne.



Dremio — Virtualisation et exposition SQL des résultats DQ

Plateforme de virtualisation de données exposant les tables Hive sous forme de vues SQL. Les scores et anomalies DQ sont exposés dans le schéma `dq-tiers` et consommés directement par Tableau sans déplacement de données.



Tableau — Dashboard de suivi de la qualité des données

Outil de visualisation connecté à Dremio via ODBC. Permet aux Data Stewards et Data Owners du domaine Tiers de suivre l'évolution des scores DQ KYC/CSP, de visualiser les anomalies et de prioriser les corrections dans le système NOVA.

3.2 Catalogue des règles de Data Quality

3.2.1 Principes de conception des règles

Les règles de Data Quality ont été définies selon les principes suivants :

- **Fondement métier** : chaque règle est justifiée par un besoin métier concret (conformité réglementaire KYC, fiabilité du scoring CSP, détection d'anomalies de saisie).
- **Périmètre RAW** : toutes les règles s'appliquent sur les tables de la **zone RAW Hive** (base `raw_nova_referentietiers`) qui sont la copie conforme des tables `CCM_*` d'Oracle NOVA.
- **Seuils quantifiés** : chaque règle dispose d'un seuil d'alerte mesurable (ex : taux de valeurs nulles $> 5\%$ = alerte).
- **Criticité graduée** : chaque règle est classée selon quatre niveaux de criticité : CRITIQUE, HAUTE, MOYENNE, FAIBLE.
- **Action corrective définie** : chaque règle est associée à une action corrective à destination des Data Stewards pour correction dans NOVA.

3.2.2 Règles DQ sur les colonnes KYC

Les règles KYC portent sur les données d'état civil et de conformité réglementaire. La table RAW cible est `raw_nova_referentieltiers.ccm_npdescription`.

TABLE 3.2 – Règles DQ — Colonnes KYC

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action cor- rective
<code>name</code>	Complétude	0 valeur nulle tolérée	CRITIQUE	Blocage saisie NOVA — au- dit dossiers
<code>first_name</code>	Complétude	0 valeur nulle tolérée	CRITIQUE	Blocage saisie NOVA — au- dit dossiers
<code>birthdate</code>	Complétude	0 valeur nulle tolérée	CRITIQUE	Compléter via NOVA avec pièce d'identité
<code>birthdate</code>	Validité	Taux dates au 01/01 < 15%	HAUTE	Flagger <code>date_naissance_approx</code> — ne pas uti- liser pour le scoring
<code>civility</code>	Validité	Valeurs IN (M., MME, MLLE) uni- quement	HAUTE	Corriger va- leurs hors référentiel dans NOVA
<code>gender</code>	Validité	Valeurs IN (M, F) uni- quement	HAUTE	Corriger va- leurs hors référentiel dans NOVA
<code>gender</code> + <code>civility</code>	Cohérence	Genre M $\neq$ MME/MLLE ; Genre F $\neq$ M.	CRITIQUE	Audit croisé — correction manuelle dans NOVA
<code>nationality_coun- try</code>	Complétude	0 valeur nulle tolérée	HAUTE	Compléter via NOVA avec pièce d'identité
<code>fatca_identifica- tion_number</code>	Conformité	Clients US ou double- nationalité US : 0 FATCA null	CRITIQUE	Remonter immédiata- ment à la Conformité CIH

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
birth_city	Validité	100% des codes résolus dans le référentiel villes	MOYENNE	Mapper codes numériques vers libellés via table référentiel villes BAM/CIH

3.2.3 Règles DQ sur les colonnes CSP

Les règles CSP portent sur les données socio-professionnelles. La table RAW cible est `raw_nova_referentiel_tiers.ccm_job`.

TABLE 3.3 – Règles DQ — Colonnes CSP

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
professional_sector	Complétude	Taux NULL < 5%	HAUTE	Compléter via NOVA — agences prioritaires
occupation	Complétude	Taux NULL < 5%	HAUTE	Compléter via NOVA
socioprofessionnelle	Complétude	Taux NULL < 5%	HAUTE	Compléter via NOVA
professional_situation	Validité	Valeurs IN (SAL, FON, TRI, AUT, EMP, NDE)	HAUTE	Corriger valeurs hors référentiel
employer	Validité	Taux valeurs parasites (AUTRE, LUI MEME...) < 30%	HAUTE	Normaliser : mapper 'LUI MEME' → TRI, 'AUTRE' → NULL
Triple CSP	Cohérence	Secteur + Profession + CSP non contradictoires	HAUTE	Vérifier cohérence et corriger dans NOVA
ordnum (multi-emplois)	Unicité	Doublons avec main=0 documentés	MOYENNE	Sélectionner emploi principal (main=1) dans la jointure CIHONE
politically_exposed	Complétude	0 valeur nulle tolérée	CRITIQUE	Demander déclaration PPE à l'agence

3.2.4 Règles DQ Pipeline (Fraîcheur)

Colonne(s)	Dimension	Règle / Seuil	Criticité	Action corrective
batch_datetime_eaac4	Fraîcheur	Retard < 26h depuis dernière exécution	HAUTE	Vérifier DAG Airflow et logs Sqoop
code_segment	Cohérence	0 client sans segment valide	HAUTE	Assigner segment par défaut selon code_marche

TABLE 3.4 – Règles DQ — Fraîcheur et Pipeline

3.3 Modèle de données DQ (Persistance)

3.3.1 Base Hive dq_results

Nous créons une base Hive dédiée `dq_results` pour stocker les résultats des contrôles DQ. Elle est distincte des bases métier (`novatiers`, `staging_nova_referentieltiers`) pour garantir la séparation des responsabilités.

3.3.2 Table dq_tiers_scores — Historique des scores

Cette table enregistre, pour chaque exécution quotidienne, le résultat agrégé de chaque règle de contrôle.

Colonne	Type	Description
run_date	DATE (partition)	Date d'exécution du contrôle DQ
dag_name	STRING	Nom du DAG déclencheur (dailyow22)
table_name	STRING	Table Hive contrôlée
column_name	STRING	Colonne contrôlée
check_name	STRING	Nom descriptif de la règle DQ
dimension	STRING	Dimension DQ : completeness, validity, consistency, freshness
criticite	STRING	CRITIQUE / HAUTE / MOYENNE / FAIBLE
domaine	STRING	KYC / CSP / PIPELINE
metric_value	DOUBLE	Valeur calculée (ex : 0.023 = 2,3% de nulls)
threshold	DOUBLE	Seuil d'alerte configuré
passed	BOOLEAN	TRUE si le check est passé
score_dq	DOUBLE	Score normalisé 0–100 pour ce check

TABLE 3.5 – Modèle de la table `dq_results.dq_tiers_scores`

3.3.3 Table `dq_tiers_anomalies` — Détail des anomalies

Cette table enregistre le détail de chaque enregistrement client en situation d'anomalie, pour permettre aux Data Stewards de les identifier et de les corriger dans NOVA.

Colonne	Type	Description
<code>run_date</code>	DATE (partition)	Date d'exécution
<code>table_name</code>	STRING	Table source de l'anomalie
<code>column_name</code>	STRING	Colonne en anomalie
<code>check_name</code>	STRING	Règle DQ violée
<code>identifiant</code>	STRING	UUID du tiers en anomalie
<code>radical</code>	STRING	Numéro client CIH (clé bancaire)
<code>anomaly_detail</code>	STRING	Description de l'anomalie (ex : genre=M, civilite=MME)
<code>criticite</code>	STRING	Criticité de la règle

TABLE 3.6 – Modèle de la table `dq_results.dq_tiers_anomalies`

3.3.4 Partitionnement et format de stockage

Les deux tables sont **partitionnées par `run_date`** (DATE), ce qui permet :

- Une interrogation performante par période (dernier jour, 7 derniers jours, etc.)
- Une gestion du cycle de vie des données (rétention configurable)
- Un rechargement partiel en cas d'erreur sans affecter l'historique

Le format de stockage retenu est **ORC** (Optimized Row Columnar), cohérent avec les autres tables du Data Lake CIH Bank, pour une compression et des performances de requêtage optimales.

3.4 Conception du moteur DQ — Soda Core

3.4.1 Architecture du script `dq_tiers_checks.py`

Le script Python principal est organisé en trois phases :

1. **Initialisation** : connexion à la source de données Hive via Soda Core, chargement des fichiers de configuration YAML des règles.
2. **Exécution des checks** : Soda Core exécute les règles définies dans les fichiers YAML sur les tables Hive cibles et collecte les résultats.
3. **Persistence** : les résultats sont formatés et écrits dans les tables `dq_tiers_scores` et `dq_tiers_anomalies` via une SparkSession.

3.4.2 Fichiers de configuration YAML

Les règles DQ sont externalisées dans des fichiers YAML séparés par domaine, ce qui facilite leur maintenance sans modifier le code Python. Nous prévoyons trois fichiers :

- `checks_kyc.yml` : règles sur les colonnes KYC (CCM_NPDESCRIPTION)
- `checks_csp.yml` : règles sur les colonnes CSP (CCM_JOB)
- `checks_pipeline.yml` : règles de fraîcheur et cohérence pipeline

3.5 Conception de l'orchestration Airflow

3.5.1 Intégration dans le DAG dailyow22

La tâche DQ s'insère dans le DAG `dailyow22` existant, en **aval** de toutes les tâches d'ingestion des tables `CCM_*`. Elle ne bloque pas le pipeline en cas d'erreur (mode `TriggerRule.ALL_DONE`) afin de garantir que les résultats DQ sont produits même si certaines tables sont partiellement chargées.

La figure 3.2 illustre le positionnement de la tâche DQ dans le graphe de dépendances du DAG.

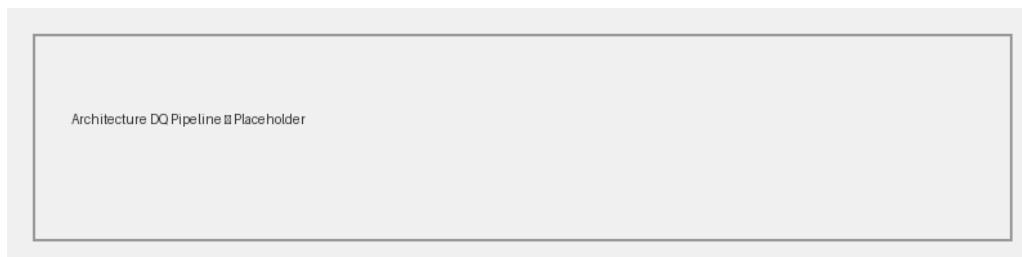


FIGURE 3.2 – Positionnement de la tâche DQ dans le DAG `dailyow22`

3.6 Conception des vues Dremio DQ

Un nouveau schéma Dremio `dq-tiers` est créé pour exposer les résultats DQ. Il comprend trois vues :

Vue Dremio	Source Hive	Usage
<code>v.dq_scores_history</code>	<code>dq_tiers_scores</code>	Historique des scores — courbes de tendance Tableau
<code>v.dq_kpis_summary</code>	<code>dq_tiers_scores</code> (agrégé)	KPIs globaux KYC/CSP par dimension et par jour
<code>v.dq_anomalies_detail</code>	<code>dq_tiers_anomalies</code>	Détail des clients en anomalie — liste pour Data Stewards

TABLE 3.7 – Vues Dremio du schéma `dq-tiers`

3.7 Conception du dashboard Tableau

Le dashboard Tableau « **Qualité Données Référentiel Tiers** » est conçu pour deux profils d'utilisateurs :

- **Data Owners et Direction** : vue exécutive avec scores globaux et alertes critiques
 - **Data Stewards** : vue opérationnelle avec liste détaillée des anomalies à corriger
- Le dashboard est structuré en **quatre onglets** :

1. **Vue Exécutive** : Score DQ global KYC + CSP du jour, évolution sur 30 jours, alertes critiques en rouge.
2. **Détail KYC** : Score par colonne KYC, heatmap par dimension DQ (Complétude / Validité / Cohérence), filtres par agence de rattachement.

3. **Détail CSP** : Score CSP, analyse des valeurs parasites dans **employer**, taux de nullité par colonne, tendance d'amélioration après corrections.
4. **Anomalies & Remédiation** : Liste paginée des tiers en anomalie, triée par criticité, exportable pour les Data Stewards. Lien direct vers le **radical** CIH pour investigation dans NOVA.

Conclusion

Ce chapitre a présenté la conception complète de la solution de Data Quality Tiers. Nous avons défini l'architecture globale à six composantes, le catalogue des règles DQ par domaine (KYC, CSP, Pipeline), le modèle de données de persistance dans Hive, la logique d'orchestration Airflow, les vues Dremio d'exposition et la structure du dashboard Tableau. Le chapitre suivant décrit la réalisation concrète de ces différents composants.

Chapitre 4

Réalisation de la Solution

Introduction

Ce chapitre présente la réalisation concrète du pipeline de Data Quality sur le Référentiel Tiers de CIH Bank. Tous les contrôles Soda Core s'effectuent exclusivement sur la **zone RAW** du Data Lake (base Hive `raw_nova_referentieltiers`), qui constitue la copie exacte et non transformée des tables `CCM_*` du système opérant Oracle NOVA. Nous détaillerons successivement l'environnement technique de développement, l'implémentation du script Soda Core, la création des tables Hive de persistance, l'intégration dans le DAG Airflow `dailyow22`, la mise en place des vues Dremio et la réalisation du dashboard Tableau.

4.1 Environnement technique de développement

4.1.1 Environnement Hadoop et cluster

Le développement et les tests ont été réalisés dans l'environnement Hadoop de CIH Bank. L'accès aux tables Hive et aux fichiers HDFS est effectué via les interfaces disponibles dans l'environnement Data Office, en utilisant les comptes de service existants du domaine Tiers.

Les versions des composants utilisés dans l'environnement de production sont les suivantes :

Composant	Version
Apache Hadoop (HDFS)	Cluster CIH Bank
Apache Hive	Cluster CIH Bank
Apache Spark	Cluster CIH Bank
Apache Airflow	Cluster CIH Bank
Dremio	Instance CIH Bank
Python	3.x
Soda Core	soda-core-spark
Tableau	Tableau Desktop / Server CIH Bank

TABLE 4.1 – Versions des composants de l'environnement technique

4.1.2 Installation de Soda Core

Soda Core est installé dans l'environnement Python accessible depuis les workers Air-flow. Le connecteur retenu est `soda-core-spark`, qui permet à Soda Core d'utiliser une `SparkSession` existante comme moteur d'exécution pour interroger les tables Hive de la zone RAW.

```
pip install soda-core-spark
```

4.2 Implémentation du moteur DQ — Soda Core

4.2.1 Structure des fichiers du projet DQ

Le projet DQ est organisé selon l'arborescence suivante :

```
dq_tiers/
|-- dq_tiers_checks.py          # Script principal Python
|-- checks/
|   |-- checks_kyc.yml          # Regles DQ colonnes KYC
|   |-- checks_csp.yml          # Regles DQ colonnes CSP
|   +-- checks_pipeline.yml     # Regles fraicheur & pipeline
+-- utils/
    +-- hive_writer.py          # Module d'écriture resultats dans Hive
```

4.2.2 Fichier de configuration YAML — Règles KYC (checks_kyc.yml)

```
# checks_kyc.yml
# Règles Data Quality - Domaine KYC
# Source : raw_nova_referentiel_tiers.ccm_npdescription (zone RAW)
# Zone RAW = copie exacte d'Oracle NOVA, aucune transformation appliquée
```

```
checks for ccm_npdescription:
```

```
# --- Complétude ---
- missing_count(name) = 0:
    name: "KYC-01 | Complétude | Nom"
    fail: error
    attributes:
        criticite: CRITIQUE
        domaine: KYC
        dimension: completeness

- missing_count(first_name) = 0:
    name: "KYC-02 | Complétude | Prénom"
    fail: error
    attributes:
        criticite: CRITIQUE
        domaine: KYC
        dimension: completeness

- missing_count(birthdate) = 0:
```

```

    name: "KYC-03 | Complétude | Date de naissance"
    fail: error
    attributes:
      criticite: CRITIQUE
      domaine: KYC
      dimension: completeness

- missing_count(nationality_country) = 0:
    name: "KYC-04 | Complétude | Nationalité"
    fail: warn
    attributes:
      criticite: HAUTE
      domaine: KYC
      dimension: completeness

# --- Validité ---
- invalid_count(civility) = 0:
    valid values: ["M.", "MME", "MLLE"]
    name: "KYC-05 | Validité | Civilité hors référentiel"
    fail: warn
    attributes:
      criticite: HAUTE
      domaine: KYC
      dimension: validity

- invalid_count(gender) = 0:
    valid values: ["M", "F"]
    name: "KYC-06 | Validité | Genre hors référentiel"
    fail: warn
    attributes:
      criticite: HAUTE
      domaine: KYC
      dimension: validity

# --- Fraîcheur ---
- freshness(batch_datetime_eaac4) < 26h:
    name: "KYC-07 | Fraîcheur | Retard batch"
    fail: error
    attributes:
      criticite: HAUTE
      domaine: PIPELINE
      dimension: freshness

```

4.2.3 Fichier de configuration YAML — Règles CSP (checks_csp.yml)

```

# checks_csp.yml
# Règles Data Quality - Domaine CSP
# Source : raw_nova_referentiel tiers.ccm_job (zone RAW)
# Zone RAW = copie exacte d'Oracle NOVA, aucune transformation appliquée

```

checks for ccm_job:

```
# --- Complétude triple CSP ---
- missing_percent(professional_sector) < 5:
    name: "CSP-01 | Complétude | Secteur professionnel"
    fail: warn
    attributes:
        criticite: HAUTE
        domaine: CSP
        dimension: completeness

- missing_percent(occupation) < 5:
    name: "CSP-02 | Complétude | Code profession"
    fail: warn
    attributes:
        criticite: HAUTE
        domaine: CSP
        dimension: completeness

- missing_percent(socioprofessional_category) < 5:
    name: "CSP-03 | Complétude | Code CSP"
    fail: warn
    attributes:
        criticite: HAUTE
        domaine: CSP
        dimension: completeness

# --- Validité ---
- invalid_count(professional_situation) = 0:
    valid values: ["SAL", "FON", "TRI", "AUT", "EMP", "NDE"]
    name: "CSP-04 | Validité | Situation professionnelle"
    fail: warn
    attributes:
        criticite: HAUTE
        domaine: CSP
        dimension: validity

# --- Conformité PPE ---
- missing_count(politically_exposed) = 0:
    name: "CSP-05 | Complétude | PPE (Personne Politiquement Exposée)"
    fail: error
    attributes:
        criticite: CRITIQUE
        domaine: CSP
        dimension: completeness
```

4.2.4 Script Python principal (dq_tiers_checks.py)

```
# dq_tiers_checks.py
# Pipeline de Data Quality - Référentiel Tiers PP Clients - Zone RAW
```



```

# CIH Bank - Data Office
# Auteur : Saad BENDAHOUE
# Périmètre : base raw_nova_referentiel_tiers (HDFS /data/raw/novatiers/)
# Déclenchement : post-ingestion DAG dailyow22 (22h00)

import argparse
from datetime import date
from pyspark.sql import SparkSession
from soda.scan import Scan

def run_dq_checks(run_date: str):
    """
    Exécute les contrôles DQ Soda Core sur les tables Hive
    du domaine Tiers et persiste les résultats.
    """
    # --- 1. Initialisation de la SparkSession ---
    spark = SparkSession.builder \
        .appName("DQ_Tiers_Checks") \
        .enableHiveSupport() \
        .getOrCreate()

    # --- 2. Initialisation du scan Soda Core ---
    scan = Scan()
    scan.set_data_source_name("hive_raw")

    # Connexion Soda --> Hive RAW via Spark
    # Base cible : raw_nova_referentiel_tiers (copie exacte NOVA)
    from soda.core.spark_df_data_source import SparkDfDataSource
    spark_source = SparkDfDataSource(
        data_source_name="hive_raw",
        spark_session=spark
    )
    scan.add_spark_session(spark, "hive_raw")

    # --- 3. Chargement des fichiers de règles YAML ---
    scan.add_sodacl_yaml_file("checks/checks_kyc.yml")
    scan.add_sodacl_yaml_file("checks/checks_csp.yml")
    scan.add_sodacl_yaml_file("checks/checks_pipeline.yml")

    # --- 4. Exécution des checks ---
    scan.execute()

    # --- 5. Collecte et persistance des résultats ---
    results = scan.get_scan_results()
    from utils.hive_writer import write_scores, write_anomalies
    write_scores(spark, results, run_date, dag_name="dailyow22")
    write_anomalies(spark, results, run_date)

    spark.stop()
    print(f"[DQ Tiers] Contrôles terminés pour la date {run_date}")

```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--run_date", type=str,
                        default=str(date.today()))
    args = parser.parse_args()
    run_dq_checks(args.run_date)

```

4.3 Création des tables Hive de persistance

Les tables de persistance DQ sont créées dans la base Hive `dq_results`. Le script HiveQL de création est le suivant :

```

-- Création de la base dédiée DQ
CREATE DATABASE IF NOT EXISTS dq_results
COMMENT 'Base de résultats Data Quality - CIH Bank Data Office';

-- Table des scores DQ historisés
CREATE TABLE IF NOT EXISTS dq_results.dq_tiers_scores (
    dag_name          STRING,
    table_name        STRING,
    column_name       STRING,
    check_name        STRING,
    dimension         STRING,
    criticite         STRING,
    domaine           STRING,
    metric_value      DOUBLE,
    threshold         DOUBLE,
    passed            BOOLEAN,
    score_dq          DOUBLE
)
PARTITIONED BY (run_date DATE)
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY');

-- Table du détail des anomalies
CREATE TABLE IF NOT EXISTS dq_results.dq_tiers_anomalies (
    table_name        STRING,
    column_name       STRING,
    check_name        STRING,
    identifier        STRING,
    radical           STRING,
    anomaly_detail    STRING,
    criticite         STRING
)
PARTITIONED BY (run_date DATE)
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY');

```

4.4 Intégration dans le DAG Airflow dailyow22

4.4.1 Ajout de la tâche DQ

La tâche de Data Quality est ajoutée au DAG `dailyow22` en tant qu'opérateur Python, déclenchée après la fin de toutes les tâches d'ingestion Sqoop des tables `CCM_*`. Le `TriggerRule.ALL_DONE` garantit que les contrôles DQ s'exécutent même si certaines tâches d'ingestion ont échoué partiellement.

```
# Extrait du DAG dailyow22 (Airflow)
from airflow.operators.python import PythonOperator
from airflow.utils.trigger_rule import TriggerRule
import subprocess

def run_dq_tiers(**context):
    run_date = context['ds'] # Format YYYY-MM-DD
    result = subprocess.run(
        [
            'python',
            '/opt/airflow/scripts/dq/dq_tiers_checks.py',
            '--run_date', run_date
        ],
        capture_output=True,
        text=True
    )
    if result.returncode != 0:
        raise Exception(f"DQ Tiers failed:\n{result.stderr}")

dq_tiers_task = PythonOperator(
    task_id='run_dq_tiers_checks',
    python_callable=run_dq_tiers,
    trigger_rule=TriggerRule.ALL_DONE,
    dag=dag
)

# Positionnement dans le graphe : après toutes les ingestions CCM_*
[ccm_party_task, ccm_npdescription_task, ccm_job_task,
 ccm_portfolio_task, ccm_context_task] >> dq_tiers_task
```

4.4.2 Planification et monitoring

Le DAG `dailyow22` s'exécute quotidiennement à **22h00**. La tâche DQ `run_dq_tiers_checks` se déclenche donc automatiquement chaque nuit après l'ingestion, sans intervention manuelle. Les logs Airflow permettent de suivre l'exécution et de détecter toute anomalie dans le pipeline DQ lui-même.

4.5 Création des vues Dremio

Trois vues SQL sont créées dans Dremio, dans le nouveau schéma `dq-tiers`, pointant vers les tables Hive de la base `dq_results`.

4.5.1 Vue v.dq_scores_history

```
-- Vue Dremio : historique des scores DQ
CREATE OR REPLACE VIEW "dq-tiers"."v.dq_scores_history" AS
SELECT
    run_date,
    dag_name,
    table_name,
    column_name,
    check_name,
    dimension,
    criticite,
    domaine,
    metric_value,
    threshold,
    passed,
    score_dq
FROM hive.dq_results.dq_tiers_scores
ORDER BY run_date DESC, criticite ASC;
```

4.5.2 Vue v.dq_kpis_summary

```
-- Vue Dremio : KPIs agrégés par domaine et dimension
CREATE OR REPLACE VIEW "dq-tiers"."v.dq_kpis_summary" AS
SELECT
    run_date,
    domaine,
    dimension,
    criticite,
    COUNT(*)                AS nb_checks,
    SUM(CASE WHEN passed THEN 1
           ELSE 0 END)      AS nb_checks_ok,
    AVG(score_dq)           AS score_moyen,
    MIN(score_dq)           AS score_min
FROM hive.dq_results.dq_tiers_scores
GROUP BY run_date, domaine, dimension, criticite
ORDER BY run_date DESC, domaine, criticite;
```

4.5.3 Vue v.dq_anomalies_detail

```
-- Vue Dremio : détail des anomalies pour Data Stewards
CREATE OR REPLACE VIEW "dq-tiers"."v.dq_anomalies_detail" AS
SELECT
    run_date,
    table_name,
    column_name,
    check_name,
    identifiant,
    radical,
    anomaly_detail,
    criticite
```

```

FROM hive.dq_results.dq_tiers_anomalies
ORDER BY run_date DESC,
        CASE criticite
            WHEN 'CRITIQUE' THEN 1
            WHEN 'HAUTE'     THEN 2
            WHEN 'MOYENNE'   THEN 3
            ELSE 4
        END;

```

4.6 Réalisation du dashboard Tableau

4.6.1 Connexion Tableau à Dremio

Le dashboard Tableau est connecté à Dremio via le connecteur ODBC/JDBC Dremio. Les trois vues du schéma `dq-tiers` sont importées comme sources de données dans Tableau : `v.dq_scores_history`, `v.dq_kpis_summary` et `v.dq_anomalies_detail`.

4.6.2 Onglet 1 — Vue Exécutive

La figure 4.1 présente l'onglet « Vue Exécutive » du dashboard.

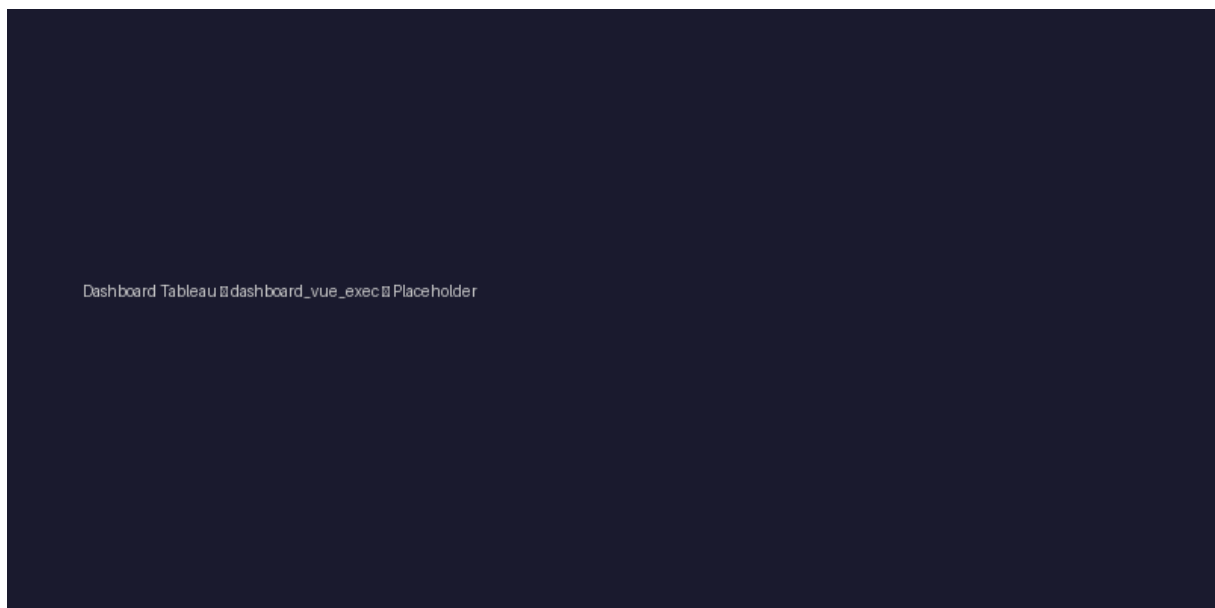


FIGURE 4.1 – Dashboard Tableau — Vue Exécutive : scores DQ globaux KYC et CSP

Cet onglet affiche :

- Le score DQ global du jour pour le domaine KYC et le domaine CSP (indicateurs chiffrés 0–100)
- La courbe d'évolution des scores sur les 30 derniers jours
- Le nombre d'alertes critiques et hautes en cours
- La date et l'heure de la dernière exécution du batch (`dailyow22`)

4.6.3 Onglet 2 — Détail KYC

La figure 4.2 présente l'onglet « Détail KYC ».

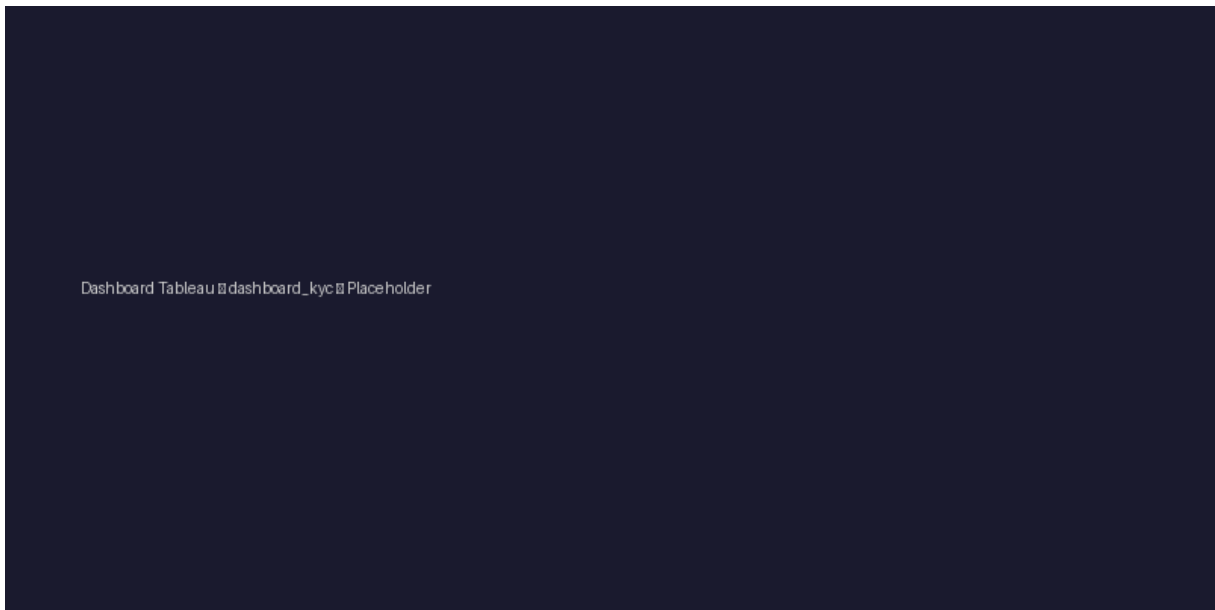


FIGURE 4.2 – Dashboard Tableau — Détail KYC : score par colonne et dimension DQ

4.6.4 Onglet 3 — Détail CSP

La figure 4.3 présente l'onglet « Détail CSP ».

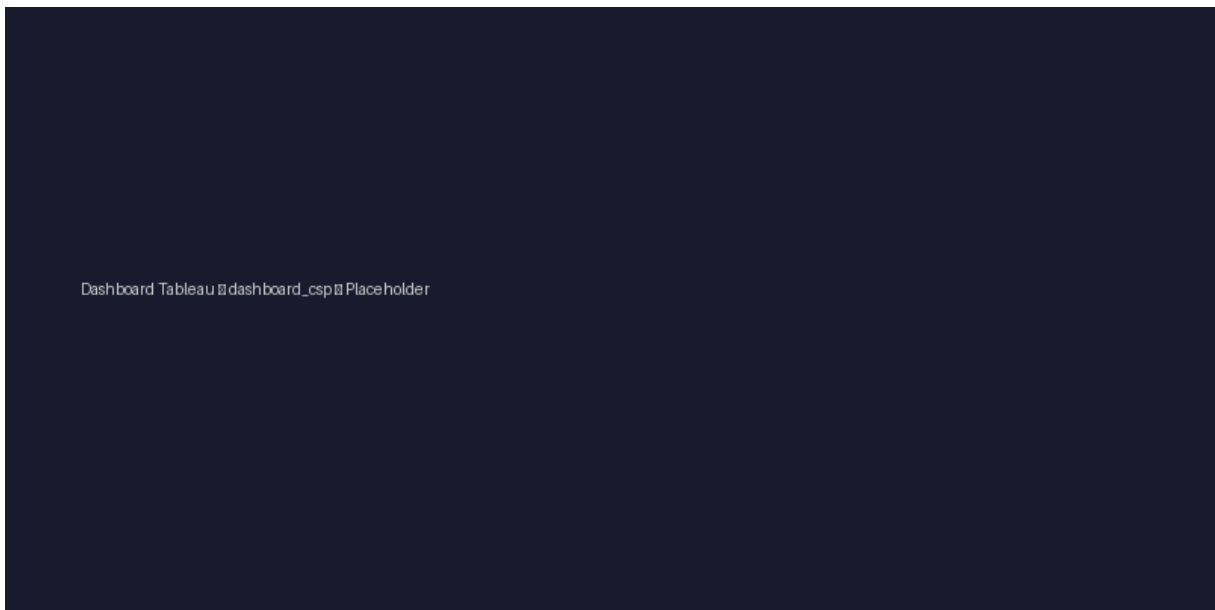


FIGURE 4.3 – Dashboard Tableau — Détail CSP : taux de nullité et valeurs parasites

4.6.5 Onglet 4 — Anomalies et Remédiation

Le quatrième onglet présente la liste paginée des enregistrements clients en anomalie, triée par criticité décroissante. Elle est directement exportable en Excel par les Data Stewards pour engagement d'actions correctives dans NOVA. Les colonnes affichées sont : `radical` (numéro client CIH), `table_name`, `column_name`, `anomaly_detail`, `criticite`, `run_date`.

Conclusion

Ce chapitre a présenté l'implémentation concrète des différents composants du pipeline de Data Quality Tiers : le script Soda Core avec ses fichiers de configuration YAML, les tables Hive de persistance, la tâche Airflow post-ingestion, les vues Dremio d'exposition et le dashboard Tableau de visualisation. L'ensemble de ces composants forme un pipeline automatisé, opérationnel chaque nuit à 22h, qui permet de mesurer, historiser et restituer la qualité des données KYC et CSP du Référentiel Tiers. Le chapitre suivant présente les résultats obtenus lors des premières exécutions. Chapitre 5 : Résultats et analyse

Conclusion générale